

DreamZero 学习笔记

-作者: Yang kunpeng

目录

第一章	DreamZero: World Action Models are Zero-shot Policies	1
1	从 VLA 到 World Action Model	1
2	模型架构: 联合视频-动作的自回归 DiT	2
2.1	为什么只让视频长期自回归	3
3	Chunk-wise Flow Matching	3
3.1	线性 probability path	3
3.2	为什么 ground-truth velocity 是端点差	3
3.3	Teacher forcing: 当前 noisy chunk 读取过去 clean chunks . .	4
3.4	联合 Flow Matching loss	5
3.5	实现中的 $w(t_k)$	5
4	闭环推理: 真实观测如何纠正视频自回归误差	6
5	实时控制: 从 5.7 s 到约 150 ms	6
6	DreamZero-Flash: 为少步动作推理重新设计训练分布	7
6.1	标准训练的 train-test mismatch	7
6.2	解耦 video 与 action timestep	7
7	把训练与推理完整串起来	8
8	与传统 VLA 的差异	9
9	值得长期保留的理解	9

DreamZero: World Action Models are Zero-shot Policies

DreamZero 最值得长期保留的不是“视频模型也能输出动作”这一表面现象，而是它把未来视觉状态与机器人动作放进同一个生成过程。视频分支描述世界接下来怎样变化，动作分支与这段视觉未来共同去噪；因此，视频预测本身就是动作推理依赖的动态表示。

这一章沿着一条完整链路来理解 DreamZero：从 VLA 与 World Action Model (WAM) 的差异开始，进入视频-动作联合概率建模、chunk-wise Flow Matching、teacher forcing、闭环 KV-cache 校正，再到 DreamZero-Flash 如何把训练噪声分布改造成适合少步实时控制的形式。

1 从 VLA 到 World Action Model

传统 VLA 更接近一个条件策略：给定当前视觉、语言和机器人状态，直接预测动作，抽象地写成

$$\pi(a \mid o, c, q).$$

这种形式擅长学习“看到什么以后应该做什么”，但并没有显式要求模型描述动作执行后世界将如何变化。

DreamZero 的出发点不同。视频生成模型已经通过大规模视频预训练学习了物体运动、接触、遮挡和场景演化等时空先验，因此更自然的问题是：**能否让模型一边想象未来，一边生成与未来一致的动作？**

设 l 为轨迹中的随机时间索引， $H > 0$ 为预测时域， $\mathbf{o}_{0:l}$ 为当前及历史视觉观测， c 为语言指令， $\mathbf{q}_l$ 为本体状态。DreamZero 联合建模

$$\pi_0(\mathbf{o}_{l:l+H}, \mathbf{a}_{l:l+H} \mid \mathbf{o}_{0:l}, c, \mathbf{q}_l). \quad (1.1)$$

这个联合分布可以按概率链式法则理解为

$$\pi_0(\mathbf{o}_{l:l+H} \mid \mathbf{o}_{0:l}, \mathbf{c}, \mathbf{q}_l) \pi_0(\mathbf{a}_{l:l+H} \mid \mathbf{o}_{0:l+H}, \mathbf{c}, \mathbf{q}_l). \quad (1.2)$$

第一项是未来视频预测；第二项在包含未来视觉的条件下恢复动作，可以理解为 inverse-dynamics model (IDM)。

真正重要的是：式 (1.2) 只是帮助理解的概率分解，DreamZero 并没有串联两个独立模型，而是用一个端到端模型同时学习未来视频和动作。因此它不是“先生成视频，再把视频交给动作头”，而是在同一个生成空间中建立 video-action 对齐。VLA 更像“从观测直接回答动作”，WAM 更像“先把未来世界与动作放在同一个概率模型里”。DreamZero 的核心迁移对象不是语言能力，而是视频模型中已经形成的时空和物理先验。

2 模型架构：联合视频-动作的自回归 DiT

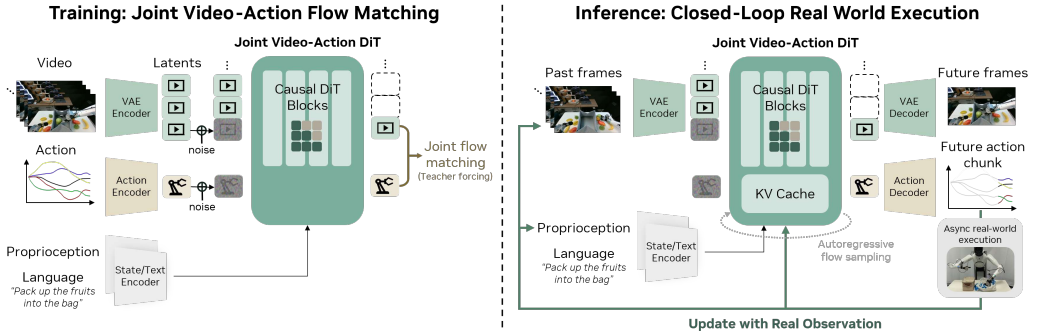


图 1.1: DreamZero 的原始模型架构。视频 latent 与动作在共享 DiT 中联合去噪；视觉历史、语言和 proprioception 作为条件参与当前 chunk 的生成。

DreamZero 从预训练 image-to-video diffusion backbone 出发，尽量减少对大视频模型本体的改造，只增加机器人相关的 state encoder、action encoder 和 action decoder。多视角机器人数据也不额外改变 backbone，而是把多个视角拼接到同一帧中。

2.1 为什么只让视频长期自回归

DreamZero 对视频采用 autoregressive chunk generation，但动作仍然按短 action chunk 生成。这一选择同时服务三个目标：

- 视频历史可以进入 KV cache，使长上下文推理更便宜；
- 下一轮生成能够利用真实视觉历史，而不是只依赖固定长度窗口；
- video frame 与 action 的时间轴能够保持更紧密的对齐。

动作不做无限期自回归，则是为了避免控制误差沿动作序列长期传播。机器人每执行完一个 action chunk，就重新读取真实环境，再决定下一段动作。

3 Chunk-wise Flow Matching

3.1 线性 probability path

对第 k 个 chunk，令 z_1^k 为干净视频 latent， a_1^k 为归一化后的干净动作。分别采样噪声

$$z_0^k \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad a_0^k \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

对去噪时间 $t_k \in [0, 1]$ ，构造线性路径

$$z_{t_k}^k = t_k z_1^k + (1 - t_k) z_0^k, \quad a_{t_k}^k = t_k a_1^k + (1 - t_k) a_0^k. \quad (1.3)$$

因此

$$t_k = 0 \Rightarrow \text{noise}, \quad t_k = 1 \Rightarrow \text{clean sample}.$$

同一个 chunk 内的视频 latent frames 与 action 共享同一个 timestep；不同 chunks 则独立采样 timestep。标准训练中，论文的最终训练算法使用 $t_k \sim \mathcal{U}(0, 1)$ 。

3.2 为什么 ground-truth velocity 是端点差

式 (1.3) 关于 t_k 是直线。直接求导：

$$\frac{dz_{t_k}^k}{dt_k} = z_1^k - z_0^k, \quad \frac{da_{t_k}^k}{dt_k} = a_1^k - a_0^k.$$

于是联合真实速度可以写成

$$\mathbf{v}^k = \begin{bmatrix} \mathbf{z}_1^k - \mathbf{z}_0^k \\ \mathbf{a}_1^k - \mathbf{a}_0^k \end{bmatrix}. \quad (1.4)$$

这里没有额外的 dynamics target，也不需要数值求解 ODE；线性 Flow Matching 的监督就是 “clean endpoint 减 noise endpoint”。

3.3 Teacher forcing: 当前 noisy chunk 读取过去 clean chunks

训练当前 chunk k 时，前面的 chunks 不再加噪，而是作为干净上下文

$$\mathcal{C}_{<k} = \bigcup_{j=1}^{k-1} \{\mathbf{z}_1^j, \mathbf{a}_1^j\}. \quad (1.5)$$

因此模型真正学习的是：在真实的历史上下文上，把当前 noisy video/action chunk 去噪成正确结果。

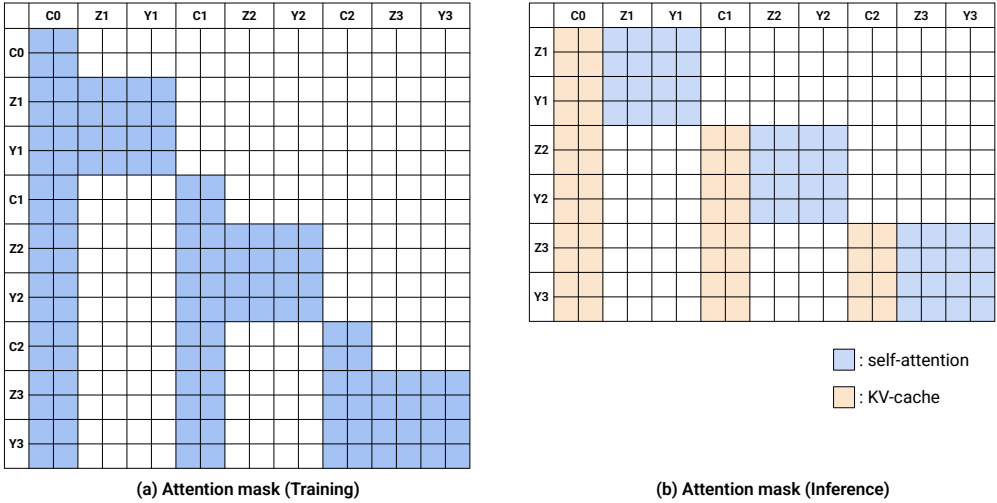


图 1.2: DreamZero 的 attention 与 KV-cache 结构。当前 noisy chunk 可以读取过去 clean context；推理时过去上下文会被真实执行后的观测刷新。

这种训练方式与机器人闭环推理形成对应关系：训练时历史是 ground-truth，部署时历史同样可以在每个 action chunk 之后由真实相机重新提供。

3.4 联合 Flow Matching loss

模型 $\mathbf{u}_\theta$ 同时预测 video 与 action 的联合 velocity。可以把训练目标写成

$$\mathcal{L}(\theta) = \mathbb{E} \left[\sum_k w(t_k) \left\| \mathbf{u}_\theta \left(\mathcal{C}_{<k}, \mathbf{z}_{t_k}^k, \mathbf{a}_{t_k}^k, \mathbf{c}, \mathbf{q}_k, \mathbf{t}_{\text{total}} \right) - \mathbf{v}^k \right\|_2^2 \right]. \quad (1.6)$$

其中 $\mathbf{t}_{\text{total}}$ 同时包含历史 clean chunks 的固定 timestep 与当前 chunk 的 t_k 。

这里最容易产生的误解是把它理解成 “video loss + action loss”。更准确的理解是：**网络直接学习一个跨两种模态的联合速度场**，video 与 action 的生成方向在同一个 DiT 表示中被共同约束。

3.5 实现中的 $w(t_k)$

论文正文只要求 $w(t_k) > 0$ ，并没有给出一个必须使用的解析函数。已有实现笔记里记录了训练代码采用的离散 scheduler 权重，这部分值得保留，因为它直接影响不同噪声阶段在训练中的相对重要性。

设 scheduler 共有 1000 个训练索引，代码先构造

$$s_i = 1 - \frac{i}{1000}, \quad \sigma_i = \frac{5s_i}{1 + 4s_i}, \quad x_i = 1000\sigma_i,$$

再定义

$$y_i = \exp \left[-2 \left(\frac{x_i - 500}{1000} \right)^2 \right],$$

最后平移并归一化得到离散权重

$$w_i = (y_i - y_{\min}) \frac{1000}{\sum_j (y_j - y_{\min})}. \quad (1.7)$$

因此 $1000^{-1} \sum_i w_i = 1$ 。这不是论文定义的新的 Flow Matching 理论，而是实现层面对不同噪声区域重新加权的具体方式。

4 闭环推理：真实观测如何纠正视频自回归误差

普通 autoregressive video generation 最大的问题是 exposure bias：前一段预测误差会变成后一段的条件，并持续累积。机器人控制拥有一个视频生成任务没有的优势：**执行动作后，真实世界会重新给出 observation。**

DreamZero 推理从联合噪声开始，在 scheduler 的多个时间点上预测联合 velocity，并逐步把 video/action chunk 从噪声推进到 clean。动作执行后，本轮预测的视频未来并不会永久写入历史，而是用相机真正看到的执行结果重新编码，并更新到 KV cache。

因此闭环可以概括成

真实观测 → 联合 video/action 去噪 → 执行 action chunk
→ 新真实观测 → 刷新 KV cache.

预测视频负责当前控制周期里的短期“想象”，真实环境负责周期性把模型重新拉回正确轨迹。DreamZero 的 stateful policy 能保留历史，但不必永久相信自己预测的未来。

5 实时控制：从 5.7 s 到约 150 ms

未经优化的 14B 视频 DiT 每个 action chunk 约需要 5.7 s，主要瓶颈包括 16 步迭代去噪、大模型前向成本和“推理完成后机器人才能继续动”的串行执行。

论文的加速不是单一技巧，而是分层处理：

层级	关键做法
执行层	异步闭环：机器人执行当前 action chunk 时，并行生成下一 chunk。
系统层	CFG conditional/unconditional 两次前向分布到不同 GPU；利用相邻 Flow velocity 的方向一致性做 DiT caching。
实现层	torch.compile、CUDA Graph、cuDNN attention、GPU scheduler、低精度量化。
模型层	DreamZero-Flash：改变 video/action timestep 的联合训练分布，使 action 支持 few-step / one-step 去噪。

异步执行把约束从“推理必须在机器人移动前结束”改成“推理必须在当前 action chunk 消耗完之前结束”。论文在双臂平台上采用 48 steps、30 Hz 的 action horizon，对应每个 chunk 约 1.6 s，因此可以让推理与动作执行充分重叠。

6 DreamZero-Flash: 为少步动作推理重新设计训练分布

6.1 标准训练的 train-test mismatch

标准 DreamZero 让 video 与 action 共用 timestep:

$$t_k^{\text{video}} = t_k^{\text{action}} = t_k, \quad t_k \sim \mathcal{U}(0, 1). \quad (1.8)$$

因此训练样本里两种模态总处于相同的噪声进度。

但 one-step / few-step 控制时真正需要的是：**action** 在很少几步内变得可执行，而 **video context** 仍可能明显带噪。如果训练阶段从未见过这种组合，直接砍掉动作去噪步数就会明显掉性能。

6.2 解耦 video 与 action timestep

DreamZero-Flash 改为

$$t_k^{\text{video}} = 1 - \eta, \quad \eta \sim \text{Beta}(\alpha, \beta), \quad t_k^{\text{action}} \sim \mathcal{U}(0, 1), \quad \alpha > \beta. \quad (1.9)$$

对应 noisy states 为

$$\begin{aligned} \mathbf{z}_{t_k^{\text{video}}}^k &= t_k^{\text{video}} \mathbf{z}_1^k + (1 - t_k^{\text{video}}) \mathbf{z}_0^k, \\ \mathbf{a}_{t_k^{\text{action}}}^k &= t_k^{\text{action}} \mathbf{a}_1^k + (1 - t_k^{\text{action}}) \mathbf{a}_0^k. \end{aligned} \quad (1.10)$$

以论文示例 $\eta \sim \text{Beta}(7, 1)$ 为例:

$$\mathbb{E}[\eta] = \frac{7}{8} = 0.875, \quad \mathbb{E}[t_k^{\text{video}}] = 0.125.$$

因为 DreamZero 的定义里 $t = 0$ 对应高噪声，video timestep 被明显推向更 noisy 的区域，而 action timestep 仍覆盖完整区间。

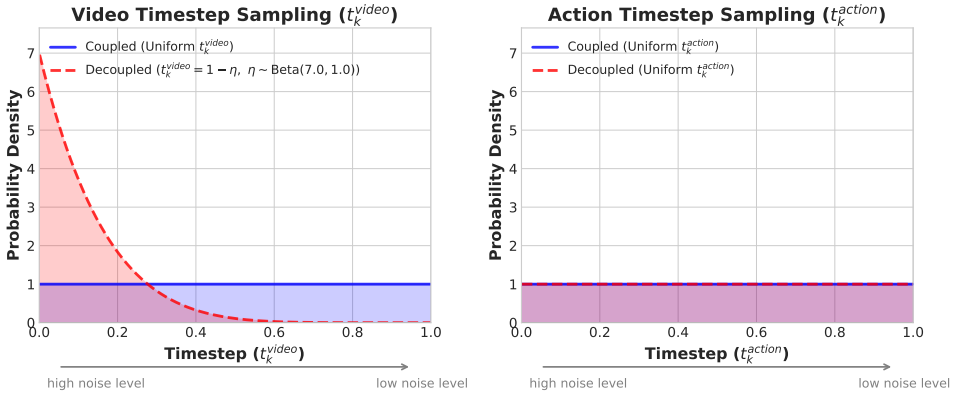


图 1.3: DreamZero-Flash 的原始 timestep 分布图。video timestep 偏向高噪声区域，action timestep 保持均匀采样。

这一步的真正作用不是“少采几个 timestep”，而是先把训练分布变成 **one-step inference** 真正会遇到的状态。论文报告，把主要 denoising steps 从 4 降到 1 时，普通 DreamZero 的任务进度从 83% 降到 52%；DreamZero-Flash 在单步下可达到 74%，推理延迟约从 350 ms 降到 150 ms。

7 把训练与推理完整串起来

DreamZero 的训练可以压缩为：

1. 把视频编码成 latent，并把动作归一化；
2. 按 chunk 划分轨迹；
3. 对当前 chunk 的 video/action 加噪，历史 chunks 保持 clean；
4. 用共享 DiT 预测联合 velocity；
5. 用式 (1.6) 更新模型，使 video 与 action 的生成方向共同对齐。

推理则是：

1. 从当前真实视觉历史、语言和 proprioception 出发；
2. 对当前 chunk 从联合噪声开始积分 Flow；
3. 得到预测 video 与 action chunk；
4. 执行动作；
5. 用真实执行后的 observation 替换预测视频历史并刷新 KV cache；

6. 进入下一控制周期。

8 与传统 VLA 的差异

维度	传统 VLA	DreamZero / WAM
主要预训练来源	图像-语言或机器人 action 数据	大规模视频生成数据
核心条件分布	$p(a \mid o, c)$	$p(o_{\text{future}}, a \mid o_{\text{history}}, c, q)$
是否显式预测未来世界	通常不需要	是，future video 与 action 联合生成
动作监督的角色	主要学习目标	与视频动态共同组成生成目标
长期历史	常用固定 observation window	自回归视频 + KV cache
闭环纠错	重新查询 policy	重新查询并把真实视觉写回世界模型历史

9 值得长期保留的理解

- **视频预测不是辅助 loss**。它是动作生成依赖的动态表示，未来世界与动作在同一个生成空间里被联合约束。
- **线性 Flow Matching 让联合训练目标非常直接**。每个 chunk 的真实 velocity 就是 clean endpoint 与 noise endpoint 的差。
- **Teacher forcing 与闭环 ground-truth feedback 是成套设计**。训练时历史 clean，部署时历史也能不断被真实 observation 刷新。
- **WAM 的泛化来源发生了变化**。它试图从互联网视频中的时空与物理先验迁移到机器人控制，而不是只依赖大规模 action-labeled robot data。
- **DreamZero-Flash 是训练分布与部署分布的对齐**。它通过解耦 video/action timestep，让模型在训练中真正见到“视觉还很 noisy，但动作必须很快 clean”的状态。

- 实时化不是只靠减少 **denoising steps**。异步执行、CFG 并行、DiT caching、kernel/quantization 与 Flash 共同组成完整的系统设计。

资料入口

- 论文: <https://arxiv.org/abs/2602.15922>
- 项目主页: <https://dreamzero0.github.io>